Tagged-Nullity Arithmetic: A Non-Contradictory System for Division by Zero with Provenance Tracking

Working paper / preprint draft

Date: 2026-11-02

Author: Alexey Dmitriyev

Contact: Lex474.by@gmail.com

Keywords: division by zero, wheel theory, meadows, IEEE 754, error provenance, debugging, computer arithmetic

Abstract

Division by zero is handled in mainstream computing by absorbing error values (NaN in IEEE 754) that destroy information about the error's origin. We propose **tagged-nullity arithmetic**: x/0 for x ≠ 0 produces a *tagged object* ⊥(x) that retains the dividend (the "tag") and its provenance history, while 0/0 = 0 by convention (following meadows). The original intuition — that the tag can be "returned" by multiplying by zero, ⊥(x)×0 = x — is shown to collapse arithmetic (it implies 2 = 1) unless associativity is abandoned, which destroys expression equivalence and testability (Richardson's theorem). We resolve this by making tag recovery a *dedicated operation* extract(⊥(x)) = x rather than multiplication. The resulting system is consistent, preserves ordinary algebra, and provides error traceability useful for debugging numerical software. A reference Python implementation with tests is included.

1. Motivation

IEEE 754 propagates NaN/±∞ silently: a long computation chain yields `inf` or nan with no indication of *where* the division by zero occurred. Existing algebraic approaches (wheel theory, transreal numbers) define 0/0 as a single absorbing element, losing the dividend. We want both: (a) a total division operation, (b) retention of the dividend, (c) compatibility with ordinary algebra.

2. Definitions

Definition 1 (Tagged nullity). For each real x ≠ 0, ⊥(x) is an object distinct from all reals, carrying *tag* x and a provenance string. ⊥(0) is identified with 0.

Notation. The primary notation is functional, ⊥(x), read "nullity of x" — by analogy with sin(x) or log(x): the parentheses carry the tag and do not affect operator precedence. For constant tags we allow the superscript shorthand $^{1}\bot \equiv \bot(1)$, $^{2}\bot \equiv \bot(2)$, etc. The functional form is essential for composite tags

(⊥(a+b), ⊥(x_1)) and for future extensions (powers: a superscript-based notation would collide with exponentiation).

Attribution of the symbol. We adopt the symbol ⊥ from wheel theory [Carlström 2004], where it denotes the single absorbing element 0/0. Our usage extends it to a *family* of tagged objects ⊥(x) — one per dividend — whereas Carlström's ⊥ carries no tag and absorbs everything. The ⊥ symbol itself is long-standing mathematical notation (falsum in logic, bottom element in lattice theory, perpendicularity in geometry) and is part of the common mathematical vocabulary (Unicode U+22A5); no claim of ownership over the symbol is made.

Axioms (System B):

- **B1.** x/0 = ⊥(x) for x ≠ 0.
- **B2.** 0/0 = 0 (meadow convention: "from preceding zero nothing larger can be obtained").
- **B3.** ⊥(a) ± ⊥(b) = ⊥(a±b).
- **B4.** a × ⊥(x) = ⊥(a·x) for real a ≠ 0.
- **B5.** ⊥(x) × 0 = 0 (multiplication by zero is *not* recovery).
- **B6.** extract(⊥(x)) = x — recovery is a *dedicated* operation, not multiplication.
- **B7.** ⊥(x) + a = ⊥(x) for real a (tag freezes when mixed with ordinary numbers).
- **B8.** ⊥(x)/0 = ⊥(x).

Convention (tag scaling under division). By linearity with B4, we adopt ⊥(x)/b = ⊥(x/b) for real b ≠ 0 (e.g., ⊥(7)/2 = ⊥(3.5)). Division **by** a nullity, in contrast, is *not* an operation of the system: defining a/⊥(x) through ordinary division would silently recover the tag, bypassing the dedicated extract and violating the principle behind B5/B6. The reference implementation rejects such division and requires the caller to apply extract explicitly — the decision to recover a value is thereby made consciously, at a well-defined point (cf. §8, open question 1).

3. The collapse theorem (why recovery cannot be multiplication)

Theorem. Any system in which (i) ⊥(x)×0 = x, (ii) multiplication is associative, and (iii) a×0 = 0 for ordinary a, satisfies 2 = 1, hence collapses to the trivial ring.

Proof.

2 = 2×1 = 2×(⊥(1)×0) [by (i)] = (2×0)×⊥(1) [by (ii)] = 0×⊥(1) [by (iii)] = ⊥(1)×0 = 1 [by (i)]. ∎

Corollary. The alternatives are exactly: (C) abandon associativity — expressions become unlabeled trees, equality checking becomes undecidable in general (Richardson 1968), and unit testing loses meaning; or (B) make recovery a non-multiplicative operation. We adopt (B).

4. Consistency of System B

Theorem. System B is consistent.

Proof (by model). Take the set $\mathbb{R} \cup (\mathbb{R}\setminus\{0\}) \times S$, where the second component stores provenance strings. Define operations by the axioms B1–B8 read left-to-right as *definitions*; each case is disjoint and total. All axioms hold by construction; ordinary reals with their operations embed as the first component. ∎

(The accompanying implementation `tagged_nullity.py` is an executable form of this model, with a test suite verifying each axiom.)

5. Debugging application

Example: f = (assets − liabilities)/(volume − hedge) × rate + fee, with volume = hedge.

- IEEE 754 (numpy): returns `inf` silently.
- System B: returns $\bot(410.4)$ with trace `(380/0)×1.08`, localizing the fault to the denominator and identifying the dividend 380.

The tag survives subsequent multiplication and addition (B4, B7), so the origin of the error is recoverable at the end of a long chain — a property neither NaN payloads (uninterpreted by the standard) nor wheel theory's single $\bot$ provide.

6. Related work

- **Wheel theory** (Carlström, 2004): $0/0 = \bot$, a single absorbing element; $x - x = 0x^2 \neq 0$. Our B3 gives $\bot(a) - \bot(a) = 0$, preserving additive cancellation; and our $\bot(x)$ is a tagged family rather than a single element (see §2, Attribution of the symbol).
- **Meadows** (Bergstra & Tucker, 2007): $0/0 = 0$ — we adopt this convention (B2); but meadows identify $x/0$ with 0 for all x, losing the dividend.
- **Transreal numbers** (Anderson): ∞ and Φ, no per-dividend tags.
- **IEEE 754 NaN payloads:** hardware carries up to 51 payload bits through NaN propagation but the standard assigns them no semantics — a natural implementation substrate for tags.

Novelty claimed: a consistent arithmetic in which the error object carries the *dividend value* plus a *provenance trace*, with recovery as a dedicated non-multiplicative operation, and with a reference implementation.

7. Implementation

See `tagged_nullity.py` (this package). Run `python3 tagged_nullity.py` to execute the test suite: axioms B1–B8, the collapse-theorem demonstration for the associative variant, the debugging scenario, and the ambiguity of the non-associative variant.

8. Open questions

1. Division *by* a nullity — deliberately excluded from the reference implementation (a silent tag recovery through ordinary division would violate the dedicated-extract principle, B5/B6); whether a safe explicit semantics exists is future work. Powers, comparisons — currently undefined or partial.

2. Tag propagation under mixed operations (B7 freezes tags) — can a richer algebra keep more provenance?

3. Hardware realization over IEEE 754 NaN payloads with zero-cost ordinary arithmetic.

4. What theorems of real analysis survive, and which require the excluded point ∞?

References

1. Carlström, J. (2004). *Wheels — On Division by Zero.* Mathematical Structures in Computer Science, 14(1), 143–184. (Extended version of Stockholm University research report 2001:11.)

2. Bergstra, J. A., & Tucker, J. V. (2007). *The rational numbers as an abstract data type.* JACM 54(2).

3. Anderson, J. A. D. W. (2006). *Perspex Machine XI: Topology of the Transreal Numbers.*

4. IEEE 754-2019. *Standard for Floating-Point Arithmetic.*

5. Richardson, D. (1968). *Some undecidable problems involving elementary functions of a real variable.* JSL 33(4).

License of this document: CC BY 4.0 (see LICENSE.txt). Code: MIT (see LICENSE.txt).